Duplicate Feature Maintenance Report

Student: Oskar / Tokushiro

Course: Software Maintenance

Project: https://github.com/Tokushiro/JHotDraw

Upstream case study: https://github.com/sweat-tek/JHotDraw

Branch: `maintenance/duplicate-feature`

Draft pull request: https://github.com/Tokushiro/JHotDraw/pull/2

CI push run: https://github.com/Tokushiro/JHotDraw/actions/runs/27103448409

CI pull-request run: https://github.com/Tokushiro/JHotDraw/actions/runs/27103453227

Date: 2026-06-07

Backlog evidence item: https://github.com/Tokushiro/JHotDraw/issues/1

1. Change Request

Requested Feature

Duplicate selected drawing elements.

User Story

As a drawing user, I want to duplicate selected drawing elements so that I can quickly reuse existing shapes without recreating them manually.

Details and Acceptance Expectations

- It must be possible to trigger the feature through the existing Duplicate edit action.
- If the active component is an enabled editable drawing component, the action must delegate to the component's `duplicate()` operation.
- Duplicated figures must preserve the source figure structure, be offset from the original selection, and become selected after the operation.
- The feature must not modify drawing state when no editable target is active.
- Existing Copy and Clear Selection action behavior must remain unchanged after refactoring shared action logic.

Repository and Environment Setup

The course lab repository was forked from `sweat-tek/JHotDraw` to `Tokushiro/JHotDraw` and cloned into:

`C:\Users\oki13\Desktop\University\System Maintainance\Working on\JHotDraw-Duplicate-Maintenance`

The work was performed on branch:

`maintenance/duplicate-feature`

The lab requested JDK 11 and Maven 3.8.x. The machine had Java 25 on PATH and no Maven on PATH, so portable tools were installed under the workspace:

- JDK: Temurin 11.0.31
- Maven: Apache Maven 3.8.8

Baseline build command:

```
mvn -s .maven-settings.xml clean install -DskipTests
```

Baseline result:

```
BUILD SUCCESS
```

GUI smoke command from `jhotdraw-samples/jhotdraw-samples-misc`:

```
mvn -s ..\..\.maven-settings.xml exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"
```

GUI smoke result:

- The process was still running after 12 seconds, which indicates that the Swing application had started and was waiting for user interaction.
- Console output was written to `report/gui-launch.log`.
- The output contained only resource icon warnings, for example missing optional `edit.duplicate.icon`, and no startup exception.

2. Concept Location

The Duplicate feature was located by searching for the `edit.duplicate` action id and calls to `duplicate()`.

Main feature entry point:

`jhotdraw-actions/src/main/java/org/jhotdraw/action/edit/DuplicateAction.java`

The action resolves the active Swing component and, when it is an enabled `EditableComponent`, delegates to:

```
((EditableComponent) c).duplicate();
```

The feature contract is defined by:

`jhotdraw-api/src/main/java/org/jhotdraw/api/gui/EditableComponent.java`

The drawing-view implementations that perform the real duplication are:

- `jhotdraw-core/src/main/java/org/jhotdraw/draw/DefaultDrawingView.java`
- `jhotdraw-core/src/main/java/org/jhotdraw/draw/AbstractDrawingView.java`

The implementation follows these steps:

1. Sort the current selected figures through the drawing.
2. Clear the current selection.
3. Clone each selected figure.
4. Translate each clone by `(5, 5)`.
5. Add the clones to the drawing.
6. Remap cloned figure relationships using the original-to-duplicate map.
7. Select the duplicated figures.
8. Register an undoable edit that removes duplicates on undo and adds them on redo.

Initial Domain Class Responsibility Table

Domain class	Responsibility in Duplicate feature
`DuplicateAction`	User-facing edit action that starts the Duplicate feature from menus/toolbars/focus context.
`AbstractSelectionAction`	Shared base class for edit actions that operate on a selected or focused component.
`EditableComponent`	Contract exposing `duplicate()`, `delete()`, `selectAll()`, `clearSelection()`, and selection state to edit actions.
`DefaultDrawingView`	Main Swing drawing view; implements `EditableComponent` and duplicates selected figures in normal drawing views.
`AbstractDrawingView`	Abstract drawing-view implementation with the same duplicate algorithm for subclasses using this view base.
`Drawing`	Container and mediator for figures; sorts selected figures, adds/removes figures, and emits undoable edit events.
`Figure`	Domain object cloned and transformed by the duplicate operation.
`AbstractFigure` and figure subclasses	Concrete figure behavior for cloning, transformation, bounds, attributes, and relationship remapping.
`DefaultApplicationModel`	Registers `DuplicateAction.ID` in the application action map.
`DefaultMenuBuilder`	Adds the Duplicate action to the Edit menu when the action exists.
SVG/ODG/PERT/NET panels and toolbars	Sample UI entry points that add Duplicate buttons/actions to user-facing sample applications.

3. Impact Analysis

The initial impact set was the code directly involved in starting and executing the Duplicate feature:

- `DuplicateAction`
- `AbstractSelectionAction`
- `EditableComponent`
- `DefaultDrawingView`
- `AbstractDrawingView`
- `Drawing`
- `Figure`

During static analysis, the feature was also found in application registration, menus, toolbar construction, and action labels. Those classes are part of the estimated impact set because a broken action id or changed action contract would affect the feature from the user interface.

Packages Visited

Package	Classes visited or counted	Contribution to feature	Change risk
`org.jhotdraw.action.edit`	13	Contains `DuplicateAction` and neighboring edit actions with duplicated active-component lookup.	Medium: user-facing commands share common behavior.
`org.jhotdraw.api.gui`	5	Defines `EditableComponent`, the action-to-view contract.	High: contract changes would ripple broadly. No contract change was made.
`org.jhotdraw.draw`	23	Contains drawing views and drawing model abstractions that perform actual duplication.	High: drawing view behavior affects many features. No change was made here.
`org.jhotdraw.draw.figure`	27	Figure domain objects are cloned, transformed, and remapped during duplication.	High: concrete figure changes can affect rendering and editing. No change was made here.
`org.jhotdraw.draw.io`	8	Input/output format strategies support clipboard and transfer behavior around drawing data.	Medium: related to copy/paste, not changed.
`org.jhotdraw.datatransfer`	10	Clipboard abstraction used by Copy; relevant because Copy shares the refactored lookup helper.	Low to medium: behavior covered by test, no production change here.
`org.jhotdraw.app`	14	Registers action ids and builds default application menus.	Low: no change needed because action id stayed stable.
`org.jhotdraw.samples.svg.gui`	33	SVG sample toolbar/menu entry points expose Duplicate in the lab GUI.	Low: no change needed because action construction stayed stable.

Scattering and Tangling

The Duplicate feature is scattered across action, API, drawing-view, drawing-domain, application, and sample UI packages. The action entry point is small, but the observable behavior depends on the drawing model and figure clone/remap behavior.

The most tangled areas are `org.jhotdraw.draw` and `org.jhotdraw.draw.figure`, because drawing views and figures support many other editing features. Changing the duplicate algorithm there would have a larger ripple effect. For that reason, the implementation deliberately avoided changing the domain algorithm and focused on action-layer duplication.

Estimated Impact Set

Before implementation, the expected production change set was:

Package	Expected changed classes	Reason
`org.jhotdraw.action.edit`	4	Pull duplicated active-component lookup into the common action base and update Duplicate, Copy, and Clear Selection.
`org.jhotdraw.actions` module POM	1	Add JUnit 4 for action-level tests.
`.github/workflows`	1	Add Maven CI workflow.
Root settings	1	Add Maven settings file for GitHub Packages credentials through environment variables.

4. Refactoring Patterns and Code Smells

Code Smell

The identified smell was duplicated code in the action layer. `DuplicateAction`, `CopyAction`, and `ClearSelectionAction` all repeated the same target-resolution pattern:

1. Use explicit `target` if present.
2. Otherwise ask the `KeyboardFocusManager` for the permanent focus owner.
3. Use it only if it is a `JComponent`.

This duplication is small but meaningful because edit actions are user-facing and more actions may need the same focus resolution. If the focus-resolution rule changes, duplicated copies can drift and create inconsistent action behavior.

Refactoring Pattern

The implemented refactoring is a combination of:

- Extract Method: isolate active component resolution.
- Pull Up Method: place that extracted method in `AbstractSelectionAction`.

The new shared method is:

```
protected JComponent getActiveComponent()
```

It returns the explicit target when present. Otherwise, it returns the permanent focus owner only when it is a `JComponent`.

Rationale

The refactoring keeps behavior unchanged while improving maintainability:

- Duplicate, Copy, and Clear Selection no longer duplicate focus lookup logic.
- `DuplicateAction.actionPerformed()` now reads as Duplicate-specific logic rather than plumbing.
- Copy can still act on disabled components because `CopyAction` still does not check `isEnabled()`.
- Clear Selection still supports both `EditableComponent` and `JTextComponent`.
- Delete was intentionally not changed because it extends `TextAction` and has a separate text deletion path.

5. Refactoring Implementation

Changed Production Classes

File	Change
`AbstractSelectionAction.java`	Added `getActiveComponent()` helper.
`DuplicateAction.java`	Replaced duplicated focus lookup with `getActiveComponent()`.
`CopyAction.java`	Replaced duplicated focus lookup with `getActiveComponent()` while preserving disabled-copy behavior.
`ClearSelectionAction.java`	Replaced duplicated focus lookup with `getActiveComponent()`.
`jhotdraw-actions/pom.xml`	Added JUnit 4.13.2 as a test dependency.

Added Support Files

File	Purpose
`.maven-settings.xml`	Allows Maven to authenticate to GitHub Packages using `GITHUB_ACTOR` and `GITHUB_TOKEN` environment variables.
`.github/workflows/maven.yml`	Runs Maven build/tests with Temurin JDK 11 for pushes and pull requests.
`DuplicateActionTest.java`	JUnit 4 tests for Duplicate, Clear Selection, and Copy behavior around the refactored helper.
`report/gui-launch.log`	Evidence from the SVG GUI smoke launch.
`report/duplicate-maintenance-report.md`	This report.

Estimate vs. Reality

The implementation matched the estimate closely.

Category	Estimated	Actual
Production Java classes changed	4	4
POM files changed	1	1
Test classes added	1	1
CI/settings files added	2	2
Drawing-domain classes changed	0	0

The most important actualization decision was to leave `DefaultDrawingView`, `AbstractDrawingView`, `Drawing`, and `Figure` untouched. That avoided unnecessary risk in the highly tangled drawing domain.

Clean Code, Clean Architecture, and SOLID Context

Clean Code:

- The action methods now contain less repeated plumbing and more feature-specific intent.
- The helper name `getActiveComponent()` describes the shared concept directly.
- Duplicate, Copy, and Clear Selection are easier to compare because their common setup code has been removed.

Clean Architecture:

- The action layer remains an outer UI/control layer.
- The domain behavior remains in drawing views and figures.
- No dependency direction was inverted or worsened; the action still depends on the `EditableComponent` abstraction rather than concrete drawing classes.

SOLID:

- Single Responsibility Principle: active component lookup is now the responsibility of the shared action abstraction, not each concrete action.
- Open/Closed Principle: future selection actions can reuse the helper without rewriting lookup logic.
- Liskov Substitution Principle: concrete actions remain valid subclasses of `AbstractSelectionAction` and keep their existing action behavior.
- Interface Segregation Principle: `EditableComponent` was not expanded; tests use only the existing contract.
- Dependency Inversion Principle: `DuplicateAction` still depends on `EditableComponent`, not on `DefaultDrawingView`.

6. Verification

Automated Tests Added

Test class:

`jhotdraw-actions/src/test/java/org/jhotdraw/action/edit/DuplicateActionTest.java`

Test cases:

Test	Verifies
`duplicateDelegatesToEnabledEditableTarget`	Duplicate calls `duplicate()` exactly through the editable target and not unrelated edit methods.
`duplicateIgnoresDisabledEditableTarget`	Duplicate does not act on disabled editable targets.
`clearSelectionDelegatesToEnabledEditableTarget`	Clear Selection still delegates to `clearSelection()`.
`clearSelectionCollapsesTextSelection`	Clear Selection still supports `JTextComponent`.
`copyStillExportsDisabledTargetToClipboard`	Copy still exports from disabled targets after helper extraction.

Commands and Results

Portable tool verification:

```
Apache Maven 3.8.8
Java version: 11.0.31, vendor: Eclipse Adoptium
```

Module test:

```
mvn -s .maven-settings.xml -pl jhotdraw-actions test
```

Result:

```
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Full reactor test:

```
mvn -s .maven-settings.xml test
```

Result:

```
BUILD SUCCESS
```

Full clean install:

```
mvn -s .maven-settings.xml clean install
```

Result:

```
BUILD SUCCESS
```

Test coverage is focused on the refactoring surface. It does not deeply retest the figure-cloning algorithm because that algorithm was intentionally not changed.

7. BDD Scenarios

JGiven was not added because the useful automation surface for this refactoring is small and already covered by JUnit 4. The BDD scenarios are still mapped formally from the user story and can be automated later with JGiven stages if required.

Scenario 1: Duplicate selected drawing elements

Given I have selected one or more figures in an enabled drawing view

When I trigger the Duplicate action

Then the drawing view duplicates the selected figures

And the duplicated figures are offset from the originals

And the duplicated figures become the selected figures

And an undoable edit is registered for the operation

Scenario 2: Ignore disabled target

Given the Duplicate action has a disabled editable target

When I trigger the Duplicate action

Then the target's `duplicate()` operation is not called

And no drawing state is changed

Scenario 3: Preserve neighboring edit action behavior

Given Copy and Clear Selection share the same active-component lookup mechanism

When Copy is triggered on a disabled target

Then Copy still exports through the target transfer handler

When Clear Selection is triggered on a text component

Then the selected text range is collapsed

8. Conclusion

The Duplicate feature was selected, located, analyzed, refactored, and verified according to the software maintenance process from the course:

Initiation -> Concept Location -> Impact Analysis -> Prefactoring -> Actualization -> Postfactoring -> Conclusion.

The implemented maintenance change improves the action layer by removing duplicated focus-resolution logic from three edit actions and centralizing it in `AbstractSelectionAction`. The observable behavior of Duplicate, Copy, and Clear Selection is preserved by focused JUnit tests and full Maven verification.

The final change is intentionally low risk:

- No public action ids changed.
- No `EditableComponent` contract changed.
- No drawing-domain duplication algorithm changed.
- No sample GUI registration changed.
- CI was added so future pull requests build and test with JDK 11.

Remaining risk:

- GitHub Projects could not be used directly because the current GitHub CLI token was missing project scopes. A GitHub issue was created as a backlog evidence item instead.
- CI run evidence is available from the successful push and pull-request workflow runs linked at the top of this report.
- JGiven automation was not added; BDD scenarios are documented and the refactoring behavior is covered by JUnit 4.

Future maintenance recommendation:

Review other edit actions such as Select All, Cut, Paste, and Delete for similar active-component lookup duplication. Delete should be handled carefully because it extends `TextAction` and includes text-specific deletion behavior.